Assignment No. 1 (Group A)

Title: Design a distributed application using RPC for remote computation where client submits an integer value to the server and server calculates factorial and returns the result to the client program.

Aim: Design a distributed application using RPC for remote computation where client submits an integer value to the server and server calculates factorial and returns the result to the client program.

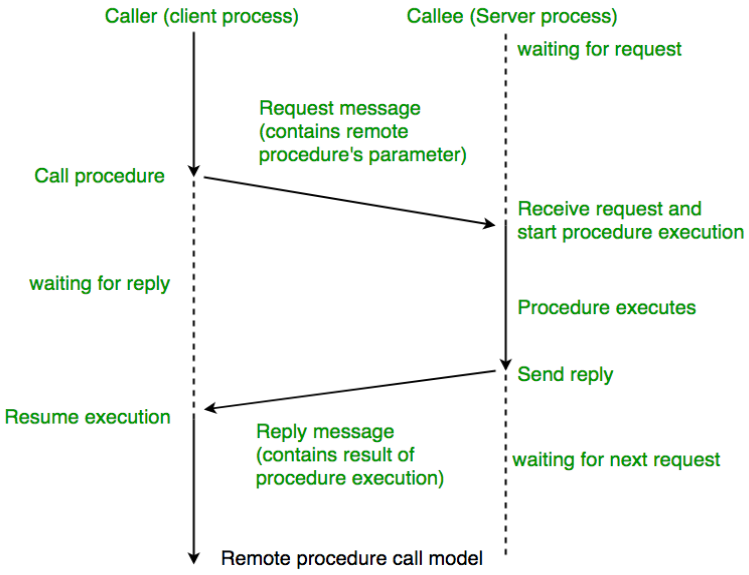
- **Outcome:** At end of this experiment, student will be able understand remote Procedure call
- **Hardware Requirement:** Computer System, Linux(Ubantu)
- **Software Requirement:** Java(JDK) & PyCharm IDE

Theory:

Remote Procedure Call (RPC) is a powerful technique for constructing distributed, client-server based applications. It is based on extending the conventional local procedure calling so that the called procedure need not exist in the same address space as the calling

procedure. The two processes may be on the same system, or they may be on different systems with a network connecting them.

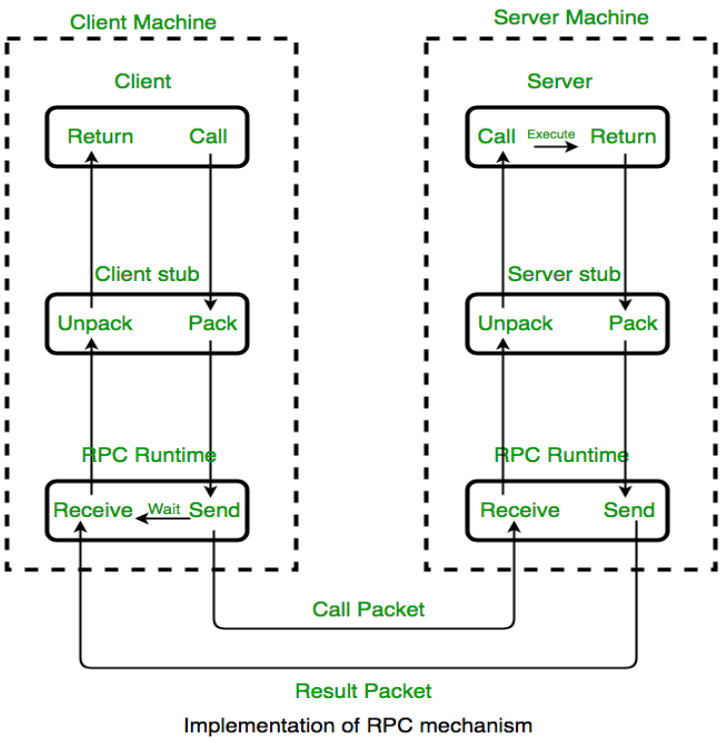
When making a Remote Procedure Call:



1. The calling environment is suspended, procedure parameters are transferred across the network to the environment where the procedure is to execute, and the procedure is executed there.

2. When the procedure finishes and produces its results, its results are transferred back to the calling environment, where execution resumes as if returning from a regular procedure call.

> **NOTE: RPC is especially well suited for client-server (e.g. query-response) interaction in which the flow of control alternates between the caller and callee. Conceptually, the client and server do not both execute at the same time. Instead, the thread of execution jumps from the caller to the callee and then back again**



Implementation of RPC mechanism

Working Of RPC:

The following steps take place during a RPC :

A client invokes a client stub procedure, passing parameters in the usual way. The client stub resides within the client's own address space.

The client stub marshalls(pack) the parameters into a message. Marshalling includes converting the representation of the parameters into a standard format, and copying each parameter into the message.

The client stub passes the message to the transport layer, which sends it to the remote server machine.

On the server, the transport layer passes the message to a server stub, which demarshalls(unpack) the parameters and calls the desired server routine using the regular procedure call mechanism.

When the server procedure completes, it returns to the server stub (e.g., via a normal procedure call return), which marshalls the return values into a message. The server stub then hands the message to the transport layer.

The transport layer sends the result message back to the client transport layer, which hands the message back to the client stub.

The client stub demarshalls the return parameters and execution returns to the caller.

Key Considerations for Designing and Implementing RPC Systems are:

Security: Since RPC involves communication over the network, security is a major concern. Measures such as authentication, encryption, and authorization must be implemented to prevent unauthorized access and protect sensitive data.

Scalability: As the number of clients and servers increases, the performance of the RPC system must not degrade. Load balancing techniques and efficient resource utilization are important for scalability.

Fault tolerance: The RPC system should be resilient to network failures, server crashes, and other unexpected events. Measures such as redundancy, failover, and graceful degradation can help ensure fault tolerance.

Standardization: There are several RPC frameworks and protocols available, and it is important to choose a standardized and widely accepted one to ensure interoperability and compatibility across different platforms and programming languages.

Performance tuning: Fine-tuning the RPC system for optimal performance is important. This may involve optimizing the network protocol, minimizing the data transferred over the network, and reducing the latency and overhead associated with RPC calls.

RPC ISSUES :
Issues that must be addressed:

1. RPC Runtime:
RPC run-time system is a library of routines and a set of services that handle the network communications that underlie the RPC mechanism. In the course of an RPC call, client-side and server-side run-time systems' code handle binding, establish communications over an appropriate protocol, pass call data between the client and server, and handle communications errors.

2. Stub:
The function of the stub is to provide transparency to the programmer-written application code.

On the client side, the stub handles the interface between the client's local procedure call and the run-time system, marshalling and unmarshalling data, invoking the RPC run-time protocol, and if requested, carrying out some of the binding steps.

On the server side, the stub provides a similar interface between the run-time system and the local manager procedures that are executed by the server.

3. Binding: How does the client know who to call, and where the service resides? The most flexible solution is to use dynamic binding and find the server at run time when the RPC is first made. The first time the client stub is invoked, it contacts a name server to determine the transport address at which the server resides.

Binding consists of two parts:
Naming:
Locating:
A Server having a service to offer exports an interface for it. Exporting an interface registers it with the system so that clients can use it.

A Client must import an (exported) interface before communication can begin.

4. The call semantics associated with RPC :
It is mainly classified into following choices-

Retry request message –
Whether to retry sending a request message when a server has failed or the receiver didn't receive the message.

Duplicate filtering –
Remove the duplicate server requests.

Retransmission of results –
To resend lost messages without re-executing the operations at the server side.

ADVANTAGES:

RPC provides ABSTRACTION i.e message-passing nature of network communication is hidden from the user.

RPC often omits many of the protocol layers to improve performance. Even a small performance improvement is important because a program may invoke RPCs often.

RPC enables the usage of the applications in the distributed environment, not only in the local environment.

With RPC code re-writing / re-developing effort is minimized.

Process-oriented and thread oriented models supported by RPC.

Conclusion:

Questions:

Q1. Explain RPC w.r.t Distributed?

Q2. Explain RPC Implementation mechanism?

Q3. Define Marshalls & DeMarshalls interms of RPC?

Q4. What Are the Issues with RPC?

Assignment No. 2 (Part-A)

Title: Design a distributed application using RMI for remote computation where client submits two strings to the server and server returns the concatenation of the given strings.

Aim: Design a distributed application using RMI for remote computation where client submits two strings to the server and server returns the concatenation of the given strings.

- **Outcome:** At end of this experiment, student will be able understand remote Procedure call
- **Hardware Requirement:** Computer System, Linux(Ubantu)
- **Software Requirement:** Java(JDK) & PyCharm IDE

Theory:

RMI (Remote Method Invocation):

The RMI (Remote Method Invocation) is an API that provides a mechanism to create distributed application in java. The RMI allows an object to invoke methods on an object running in another JVM.

The RMI provides remote communication between the applications using two objects stub and skeleton.

Understanding stub and skeleton

RMI uses stub and skeleton object for communication with the remote object.

A **remote object** is an object whose method can be invoked from another JVM. Let's understand the stub and skeleton objects:

stub

The stub is an object, acts as a gateway for the client side. All the outgoing requests are routed through it. It resides at the client side and represents the remote object. When the caller invokes method on the stub object, it does the following tasks:

1. It initiates a connection with remote Virtual Machine (JVM),
2. It writes and transmits (marshals) the parameters to the remote Virtual Machine (JVM),
3. It waits for the result
4. It reads (unmarshals) the return value or exception, and
5. It finally, returns the value to the caller.

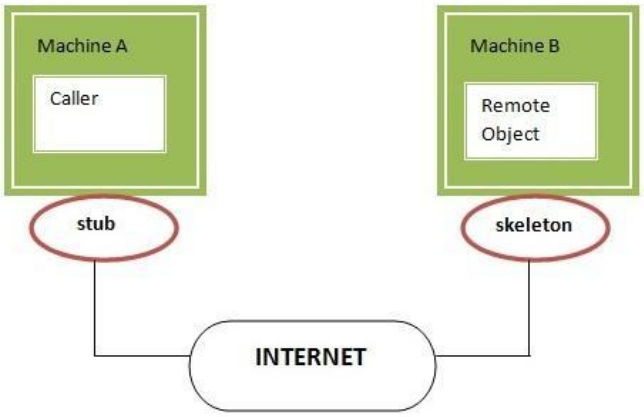
skeleton

The skeleton is an object, acts as a gateway for the server side object. All the incoming requests are routed through it. When the skeleton receives the incoming request, it does the following tasks:

It reads the parameter for the remote method

1. It invokes the method on the actual remote object, and
2. It writes and transmits (marshals) the result to the caller.

In the Java 2 SDK, an stub protocol was introduced that eliminates the need for skeletons.



Understanding requirements for the distributed applications
If any application performs these tasks, it can be distributed application.

1. The application need to locate the remote method
2. It need to provide the communication with the remote objects, and
3. The application need to load the class definitions for the objects.

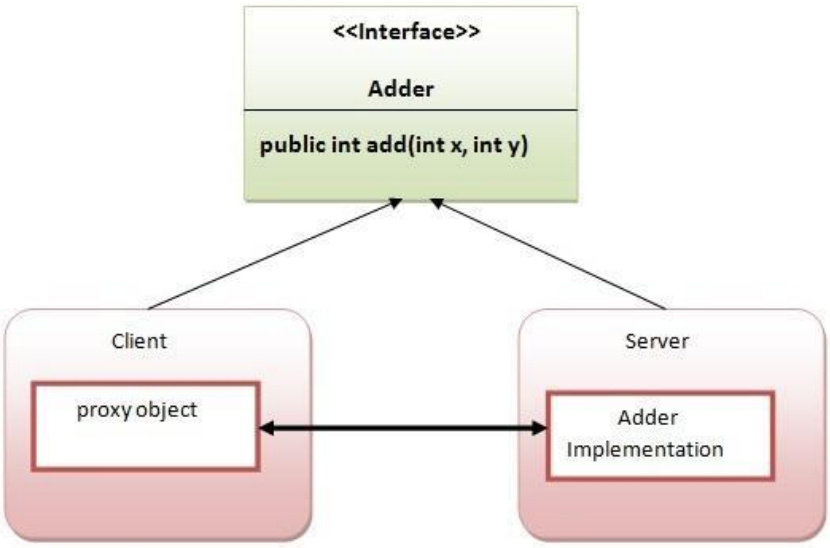
The RMI application have all these features, so it is called the distributed application.

Java RMI Example
The is given the 6 steps to write the RMI program.

1. Create the remote interface
2. Provide the implementation of the remote interface
3. Compile the implementation class and create the stub and skeleton objects using the rmic tool
4. Start the registry service by rmiregistry tool
5. Create and start the remote application
6. Create and start the client application

RMI Example

In this example, we have followed all the 6 steps to create and run the rmi application. The client application need only two files, remote interface and client application. In the rmi application, both client and server interacts with the remote interface. The client application invokes methods on the proxy object, RMI sends the request to the remote JVM. The return value is sent back to the proxy object and then to the client application



Conclusion: -

_____

_____

Questions:

1. What RMI? How its working?
2. Describe JAVA RMI as an Example?
3. What is role of stub & skeleton in RMI?

Assignment No. 3 (Part A)

Title: Implement Union, Intersection, Complement and Difference operations on fuzzy sets. Also create fuzzy relations by Cartesian product of any two fuzzy sets and perform max-min composition on any two fuzzy relations.

Aim: Implement Union, Intersection, Complement and Difference operations on fuzzy sets. Also create fuzzy relations by Cartesian product of any two fuzzy sets and perform max-min composition on any two fuzzy relations.

Title: Fuzzy set Operations & Relations

Problem Statement:

Implement Union, Intersection, Complement and Difference operations on fuzzy sets. Also create fuzzy relations by Cartesian product of any two fuzzy sets and perform max-min composition on any two fuzzy relations.

Theory:

Fuzzy Set Operations

1. Define Fuzzy Sets:

o Choose a universe of discourse (U) representing the range of possible values for your variable.

o Define two fuzzy sets, A and B, on the universe U using membership functions. Membership functions map elements in U to a degree of membership between 0 and 1. Common functions include triangular, trapezoidal, and Gaussian functions.

2. Union (OR):

o Implement a function that takes two fuzzy sets (A, B) and them membership functions (μ_A, μ_B) as input.

o For each element x in U, calculate the degree of membership in the union (μ_A U B)(x) using the maximum of individual memberships:

o μ_A U B(x) = max(μ_A(x), μ_B(x))

3. Intersection (AND):

o Implement a function that takes two fuzzy sets (A, B) and them membership functions (μ_A, μ_B) as input.

o For each element x in U, calculate the degree of membership in the intersection (μ_A ∩ B)(x) using the minimum of individual memberships:

o μ_A ∩ B(x) = min(μ_A(x), μ_B(x))

4. Complement (NOT):

o Implement a function that takes a fuzzy set (A) and its membership function (μ_A) as input.

o For each element x in U, calculate the degree of membership in the complement (μ_Ac)(x) by inverting the membership of A:

o μ_Ac(x) = 1 - μ_A(x)

5. Difference (A except B):

o Implement a function that takes two fuzzy sets (A, B) and them membership functions (μ_A, μ_B) as input.

o For each element x in U, calculate the degree of membership in the difference (μ_A \ B)(x) using the membership of A and the complement of B:

o μ_A \ B(x) = max(μ_A(x), μ_Bc(x))

Fuzzy Relations

1. Cartesian Product:

o Define two fuzzy sets, A and B, on universes U and V respectively.

o The Cartesian product of A and B creates a new fuzzy relation R on the universe U x V (cartesian product of U and V).

o The membership function of R, μ_R((u, v)), represents the degree of

relationship between element u in U and element v in V.

o Implement a function that takes two fuzzy sets (A, B) and their

membership functions (μ_A, μ_B) as input.

o For each pair (u, v) in U x V, calculate the degree of membership in the relation using the minimum of individual memberships:

o μ_R((u, v)) = min(μ_A(u), μ_B(v))

2. Max-Min Composition:

o Define two fuzzy relations, R and S, on universes (U x V) and (V x W)

respectively.

o Max-min composition combines these relations to create a new fuzzy

relation T on universe U x W.

o The membership function of T, μ_T((u, w)), represents the composed

relationship between element u in U and element w in W, considering the

relationship between u and all elements v in V.

o Implement a function that takes two fuzzy relations (R, S) and their

membership functions (μ_R, μ_S) as input.

o For each pair (u, w) in U x W, iterate through all elements v in V and

calculate the intermediate values:

o z_uv = min(μ_R((u, v)), μ_S((v, w)))

o Use the maximum of these intermediate values as the degree of

membership in the composed relation:

Conclusion:----------

--

Questions:

1. Explain how **union and intersection** of two fuzzy sets are performed. How are they different from classical set operations?
2. Given a fuzzy set A with membership values, how do you calculate its **complement**? Provide a simple example.
3. What is the **difference operation (A − B)** in fuzzy sets? How is it computed using the complement of set B?
4. Define a **fuzzy relation**. How is a fuzzy relation created using the **Cartesian product** of two fuzzy sets?
5. Explain the concept of **max-min composition** of fuzzy relations. Why do we use both *max* and *min* operations in this method?

Assignment No. 4 (Part A)

Title: Write code to simulate requests coming from clients and distribute them among the servers using the load balancing algorithms.

Aim: Write code to simulate requests coming from clients and distribute them among the servers using the load balancing algorithms.

Outcome: At end of this experiment, student will be able understand how to manage Load of Servers by balancing

- **Hardware Requirement:** Computer System, Linux(Ubuntu)
- **Software Requirement:** PyCharm IDE OR, Jupyter,Google Colab

Theory:

Introduction:
Load balancing is a crucial technique in distributed computing that ensures efficient utilization of resources by distributing incoming requests across multiple servers. It helps in preventing server overload, reducing response time, and improving system performance.

In this experiment, we implement two common load balancing algorithms:

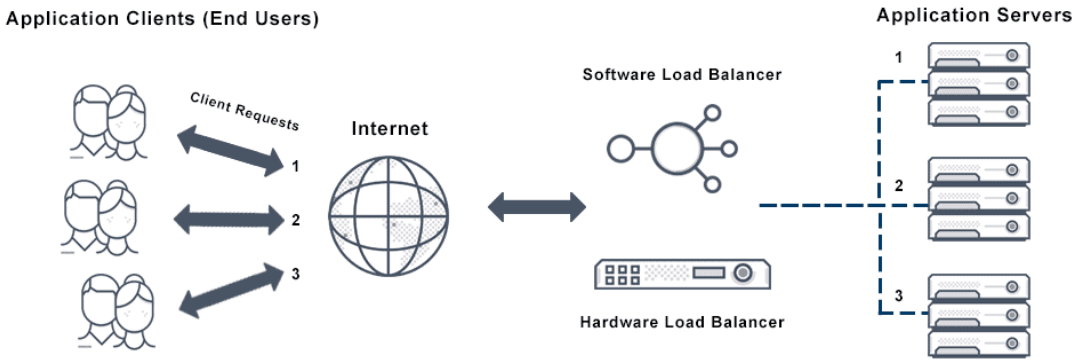
1. **Round Robin Load Balancing**
2. **Least Connections Load Balancing**

Load Balancing Algorithms:

1. **Round Robin Algorithm:**
    - Requests are assigned to servers in a cyclic order.
    - Each server gets an equal number of requests in sequence.
    - Simple to implement and effective for evenly distributed loads

How It Works: Source View

Consider a scenario with three servers: Server A, Server B, and Server C. The Round Robin algorithm assigns the first incoming request to Server A, the second to Server B, the third to Server C, and then cycles back to Server A for the fourth request. This pattern continues, rotating through the list of servers, thereby distributing the requests evenly.



Advantages:

- **Simplicity:** The algorithm is easy to implement and requires minimal computational overhead.
- **Fairness:** Each server receives an approximately equal number of requests, preventing any single server from becoming a bottleneck.

Limitations:

While Round Robin Load Balancing is effective in homogeneous environments where servers have similar capacities and the incoming requests are relatively uniform in resource demands, it has certain limitations:

- **Ignoring Server Load:** The algorithm does not consider the current load or performance capacity of each server. In cases where servers have varying processing powers or some requests require more resources than others, this can lead to inefficient load distribution.
  f5.com
- **Lack of Fault Tolerance:** If a server becomes unavailable or unresponsive, the standard Round Robin algorithm will continue to route requests to it, leading to potential failures or delays.

Variations:

To address some of these limitations, variations of the Round Robin algorithm have been developed:

- **Weighted Round Robin:** This approach assigns a weight to each server based on its capacity or performance metrics. Servers with higher weights receive a proportionally larger number of requests, allowing for better load distribution in heterogeneous environments.
  geeksforgeeks.org

Use Cases:

Round Robin Load Balancing is particularly useful in scenarios where:

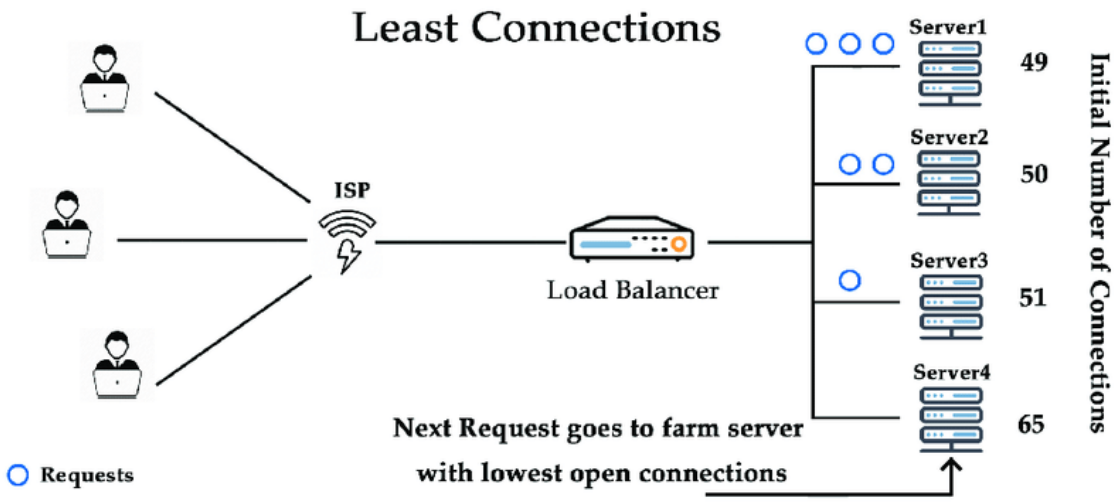
- Servers have similar specifications and capabilities.

- The workload is evenly distributed, with requests requiring comparable processing resources.
- A simple and easy-to-implement load balancing solution is desired.

2. **Least Connections Algorithm: Source View**
    - Requests are assigned to the server with the fewest active connections.
    - Ensures that no single server gets overloaded if the requests are of varying durations.
    - More dynamic and efficient in scenarios where processing times differ.

How It Works:

1. **Connection Monitoring:** The load balancer continuously tracks the number of active connections for each server in the pool.
2. **Request Allocation:** When a new request arrives, the load balancer evaluates the active connection count for each server and assigns the request to the server with the least number of active connections at that moment.



Advantages:

- **Dynamic Load Distribution:** By considering the current number of active connections, the algorithm adapts to fluctuating workloads, distributing requests based on real-time server load.
- **Improved Resource Utilization:** Ensures that all servers operate efficiently, reducing the likelihood of some servers being overburdened while others remain underutilized.

Limitations:

- **Server Capacity Ignorance:** The basic Least Connections algorithm does not account for differences in server capabilities. For instance, a high-capacity server and a low-capacity server are treated equally, which might lead to suboptimal performance.
- **Connection Persistence:** In scenarios where connections have long durations, a server might accumulate numerous connections, potentially leading to uneven load distribution over time.

Variations:

- **Weighted Least Connections:** To address differences in server capacities, this variation assigns a weight to each server based on its processing power or other performance metrics. Servers with higher weights receive a proportionally larger share of the incoming requests, balancing the load according to server capabilities.

Use Cases:

- **Heterogeneous Server Environments:** Ideal for setups where servers have varying processing powers, ensuring that more capable servers handle a larger portion of the load.
- **Variable Request Processing Times:** Suitable for applications where the duration or resource consumption of requests is unpredictable, as the algorithm dynamically adjusts to the current load on each server.

Implementation Details:

- Each server maintains an active connection count.
- The Round Robin algorithm assigns requests sequentially.
- The Least Connections algorithm assigns requests to the least loaded server.
- Requests are processed and then released to simulate real-world behavior.
- The system prints request allocation and server status after execution.

Expected Output:

- The Round Robin algorithm should distribute requests evenly across all servers.
- The Least Connections algorithm should distribute requests based on the server load, leading to a more balanced request handling process.
- The program should show step-by-step request assignments and the final load distribution.

Conclusion: -

Questions:

1. What is load balancing and why is it important?
2. Explain the working principle of the Round Robin algorithm.
3. How does the Least Connections algorithm dynamically allocate requests?
4. Which load balancing algorithm is best suited for real-time applications and why?
5. What are other load balancing techniques used in modern distributed systems?

Assignment No. 5 (Part A)

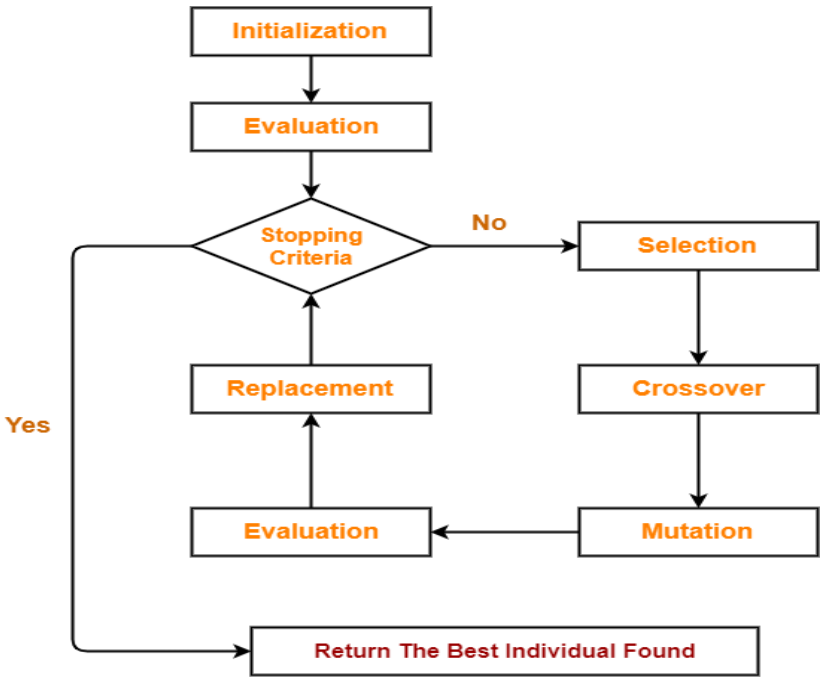
Title: Optimization of genetic algorithm parameter in hybrid genetic algorithm-neural network modelling: Application to spray drying of coconut milk.

Aim: Optimization of genetic algorithm parameter in hybrid genetic algorithm-neural network modelling: Application to spray drying of coconut milk.

- **Outcome:** At end of this experiment, student will be able understand Genetic Algorithm
- **Hardware Requirement:** Computer System, Linux(Ubantu)
- **Software Requirement:** PyCharm IDE

Theory:

Genetic Algorithm:



How Genetic Algorithm Works

- Abstraction of real biological evolution

- ❖ Solve complex Problems(NP-Hard type ex.TSP)
- ❖ Focus On Optimaization
- ❖ Population of possible solution for a given problem
- ❖ From group of individuals the best one Will survive

Genetic Algorithms offer the following advantages-

Point-01:

Genetic Algorithms are better than conventional AI.

- This is because they are more robust.

Point-02:

They do not break easily unlike older AI systems.

- They do not break easily even in the presence of reasonable noise or if the inputs get change slightly.

Point-03:

While performing search in multi modal state-space or large state-space,

Genetic algorithms has significant benefits over other typical search optimization techniques.

- GA (Genetic Algorithm) is good at taking larger, potentially huge search space and navigating them looking for optimal solution which we might not find in lifetime.
- GA is better than other traditional algorithm in that they are more robust.
- They do not break easily even if the inputs are changed slightly or in the presence of reasonable noise.

GA is used to resolve complicated optimization problems, such as , organizing the time table, scheduling job shop, playing games.

The concept of GA is directly derived from natural evolution and heredity i.e. inheritance, where child inherits the characters (stored in the chromosomes) from the parent.

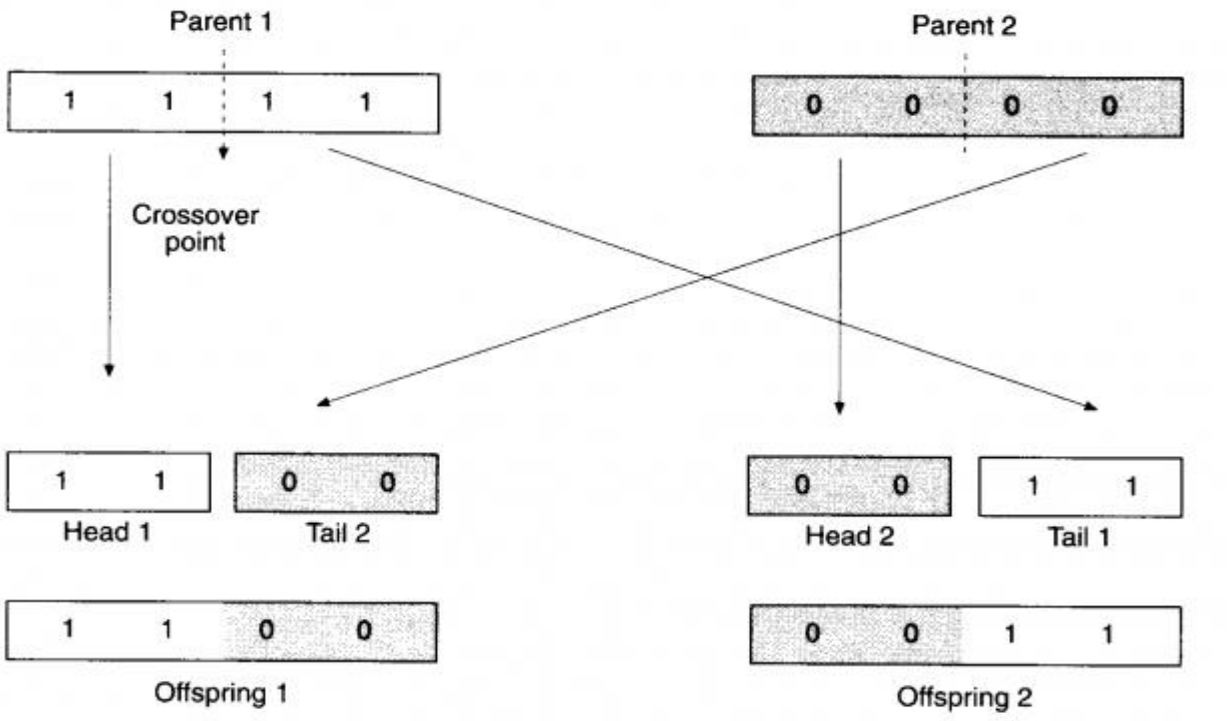
Operators in GA:

1.Crossover (Recombination):-

Crossover is the process of **taking two parent solutions and producing from them a child**. After the selection (reproduction) process, the population is enriched with better individuals. Crossover operator is applied to the mating pool with the hope that it creates a better offspring.

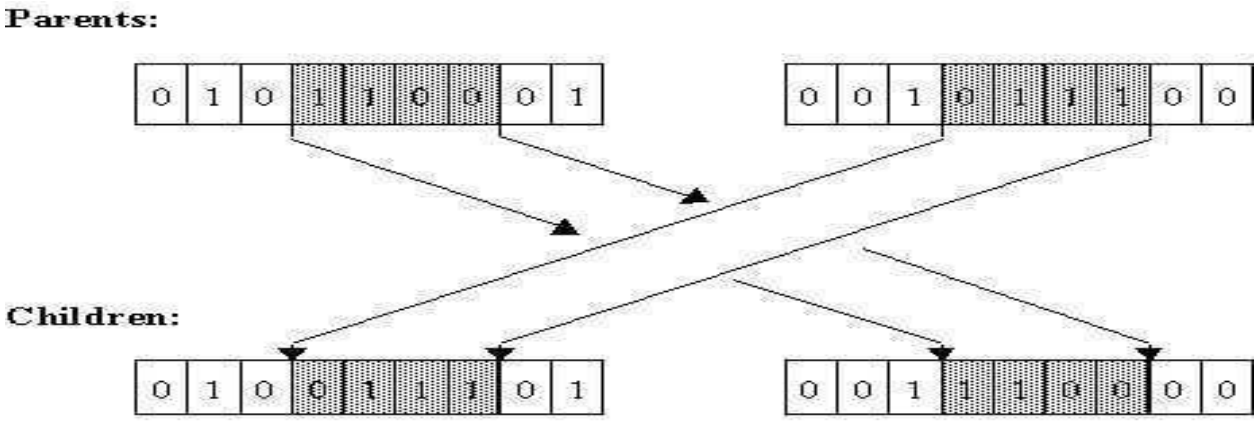
The various crossover techniques are-

i).***Single-Point Crossover-***Here the two mating chromosomes are cut once at corresponding points and the sections after the cuts exchanged.



ii). Two-Point Crossover-

Here two crossover points are chosen and the contents between these points are exchanged between two mated parents.

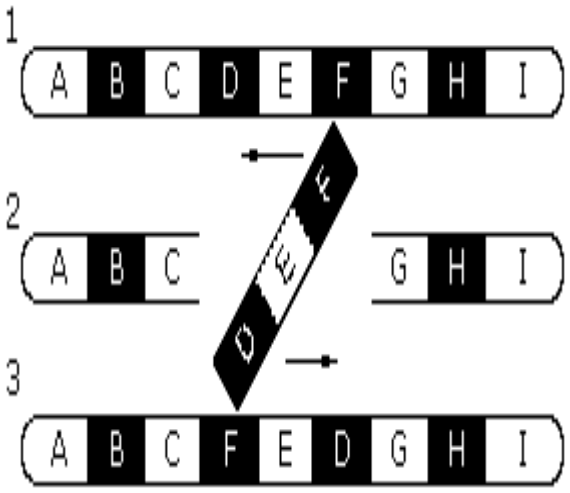


2. Inversion:-

Inversion operator inverts the bits between two random sites.

01 0011 1

Then, 0111001



3. Deletion:-

i). Deletion and Duplication-Here any two or three bits in random are selected and their previous bits are duplicated.

before duplication: 00 1001 0

deletion: 00 10_ _ 0

duplication: 00 1010 0

ii). Deletion and Regeneration-Here bits between the cross site are deleted and regenerated randomly.

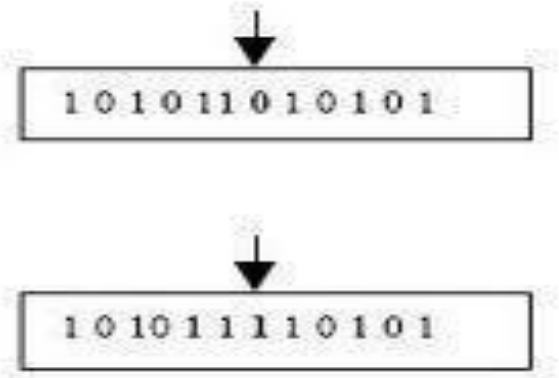
10 0110 1

10 _ _ _ _ 1

10 1101 1

4. Mutation: -

After crossover, the strings are subjected to mutation. Mutation prevents the algorithm to be trapped in a local minimum. It plays the role of recovering the genetic materials as well as for randomly distributing genetic information. It helps escape from local minima's trap and maintain diversity in the population. Mutation of a bit involves flipping a bit, changing 0 to 1and vice-versa.



Conclusion: -

_____

_____

_____

_____

____

Questions:

1. Define Phenotype & Genotype with Ex.?
2. Define Encoding & Decoding and Explain Its Technique?
3. Define Term Population, Genes, Fitness Function w.r.t Genetic Algorithm?
4. Explain Genetic algorithm with its architecture?

Assignment No. 6 ((Part A)

Title: - Implementation of Clonal selection algorithm using Python.

Aim: Implementation of Clonal selection algorithm using Python.

Theory:

Introduction:

Briefly introduce the concept of CLONALG and its inspiration from the biological immune system's clonal selection process.

Explain how CLONALG utilizes antibodies (candidate solutions) and affinity (fitness function) to iteratively improve solutions.

Core Principles:

• Antibody Representation: Define how you will represent candidate solutions in your implementation. This could be binary strings for binary optimization problems or real-valued vectors for continuous problems.

• Affinity Function: Design a function that evaluates the fitness or quality of each candidate solution (antibody) relevant to your chosen optimization problem.

CLONALG Phases:

• Initialization: Generate a random population of antibodies (candidate solutions).

• Selection: Select high-affinity antibodies (good solutions) for cloning based on their fitness function values.

• Cloning: Create clones of selected antibodies. The number of clones can be proportional to the antibody's affinity.

• Hypermutation: Introduce random mutations to the clones with a low mutation rate to promote diversity.

• Evaluation: Evaluate the affinity of all antibodies (original population and clones) using the fitness function.

• Selection and Replacement: Select the best antibodies (including original population and mutated clones) to form the next generation, replacing low-affinity ones.

• Termination: Stop the process when a termination criterion is met (e.g., reaching maximum number of iterations or achieving a desired fitness value).

Conclusion:

Questions:

1. What is the **Clonal Selection Algorithm (CLONALG)** and how is it inspired by the biological immune system?
2. Explain the concept of antibody representation and affinity function in CLONALG. How do they affect the performance of the algorithm?
3. Describe the **selection and cloning process** in CLONALG. Why are high-affinity antibodies selected for cloning?
4. What is **hyper mutation** in CLONALG? Why is it important for maintaining diversity in the population?
5. Explain all the phases of CLONALG (Initialization, Selection, Cloning, Hyper mutation, Evaluation, Replacement, Termination) in brief.

Assignment No. 7((Part A)

Title: Create and Art with Neural style transfer on given image using deep learning.

Aim: Create and Art with Neural style transfer on given image using deep learning.

Problem Statement: Create and Art with Neural style transfer on given image using deep learning.

Theory:

Introduction: Neural Style Transfer (NST):

• Neural Style Transfer is a deep learning technique that combines the content of one image (content image) with the style of another image (style image) to create a new image (generated image). • It leverages convolutional neural networks (CNNs) to extract content and style features from the input images and optimize a generated image to minimize the content distance from the content image and the style distance from the style image simultaneously.

Key Steps of NST: • Choose Content and Style Images: Select a content image (the image whose content you want to retain) and a style image (the image whose artistic style you want to apply). • Preprocess Images: Preprocess the content and style images to prepare them for feeding into a CNN. • Define Loss Functions: Define content loss and style loss functions to measure the difference between the features of the generated image, content image, and style image. • Optimization: Use gradient descent or other optimization techniques to update the generated image to minimize the combined content and style loss. • Generate Artistic Image: Generate the final artistic image by optimizing the generated image. Conclusion: Thus, we have successfully implemented a program for Neural style transfer.

Conclusion:

Questions:

1. What is **Neural Style Transfer (NST)**, and how does it create an artistic image using deep learning?
2. What is the role of **content image** and **style image** in Neural Style Transfer?
3. How are **Convolutional Neural Networks (CNNs)** used to extract content and style features in NST?
4. Explain the purpose of **content loss** and **style loss** functions in Neural Style Transfer.
5. List and explain the **key steps involved in Neural Style Transfer**, from image selection to generation of the final artistic image.

Assignment No.1 (Part B)

Title: To apply the artificial immune pattern recognition to perform a task of structure damage Classification.

Aim: To apply the artificial immune pattern recognition to perform a task of structure damage Classification

Outcome: At end of this experiment, student will be able understand AIPR

- **Hardware Requirement:** Computer System, Linux(Ubuntu)
- **Software Requirement:** PyCharm IDE OR, Jupyter

Theory:

The task at hand is to classify structural damage in various infrastructure components such as buildings, bridges, roads, etc. Given data representing different types and degrees of structural damage, the goal is to develop a classification system that can accurately identify and classify the severity of damage.

Approach using Artificial Immune Pattern Recognition (AIPR):

1. Understanding AIPR:

- AIPR is inspired by the human immune system's ability to recognize and respond to foreign pathogens.
- In AIPR, the concept of antigens and antibodies is used to detect patterns in data.
- Antigens represent patterns in the data, while antibodies are generated to recognize and classify these patterns.
- AIPR algorithms evolve a set of antibodies through a process of affinity maturation and selection to achieve accurate pattern recognition.
- 2. Data Preprocessing:
- Begin by collecting and preprocessing the data representing structural damage.
- Preprocessing may involve tasks such as normalization, feature extraction, and dimensionality reduction.

3. Representation of Data:

- Represent the structural damage data as antigens, where each antigen represents a specific pattern or feature in the data.
- Antigens can be represented as vectors or matrices depending on the nature of the data.

4. Generation of Antibodies:

- Initialize a set of antibodies representing potential classifiers.
- Antibodies are initialized randomly or using a predefined strategy.

5. Affinity Maturation:

- Apply affinity maturation to evolve the antibodies to better recognize antigens.
- During affinity maturation, antibodies undergo mutation and selection to improve their affinity towards antigens.

6. Training:

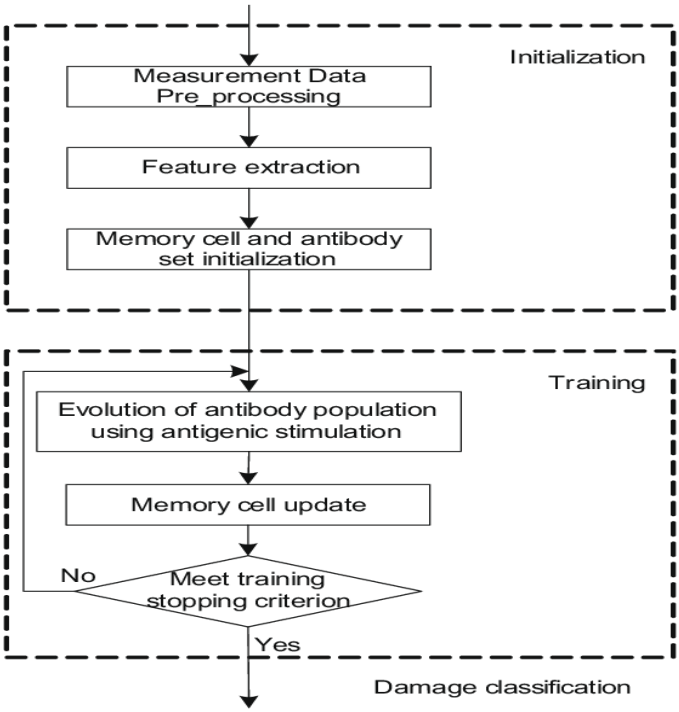
- Train the AIPR system using the preprocessed data.
- During training, the antibodies are exposed to antigens, and their affinities are adjusted based on the similarity between antibodies and antigens.

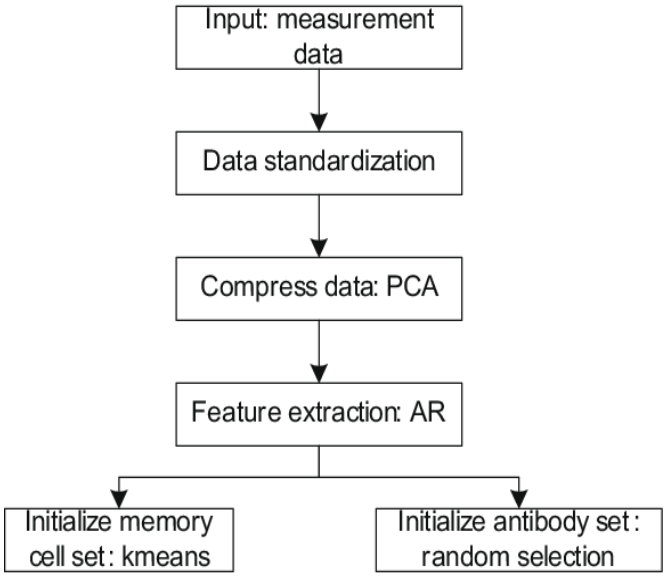
7. Classification:

- After training, the evolved antibodies can be used for classification.
- Given a new instance of structural damage, the system can classify its severity based on the affinity of antibodies towards the patterns present in the instance.

8. Evaluation:

- Evaluate the performance of the AIPR system using metrics such as accuracy, precision, recall, and F1-score.
- Fine-tune the system parameters and repeat the training and evaluation process as necessary to improve performance.





Data preprocessing in Artificial Immune Pattern Recognition (AIPR) plays a crucial role in preparing the raw data for effective pattern recognition. Here's how data preprocessing can be conducted in the context of AIPR for the task of structure damage classification:

1. **Data Cleaning:**
   - Remove any noise or irrelevant information from the dataset.
   - Handle missing values by imputation or removal.
2. **Normalization:**
   - Scale the features to a similar range to ensure that they contribute equally to pattern recognition.
   - Common normalization techniques include Min-Max scaling and Z-score normalization.
3. **Feature Extraction:**
   - Extract relevant features from the raw data that capture important characteristics related to structural damage.
   - Feature extraction techniques can include statistical measures, frequency domain analysis, or domain-specific features.
4. **Dimensionality Reduction:**
   - Reduce the dimensionality of the feature space to mitigate the curse of dimensionality and improve computational efficiency.
   - Techniques such as Principal Component Analysis (PCA) or feature selection methods can be employed for dimensionality reduction.
5. **Encoding Categorical Variables:**
   - If the dataset contains categorical variables, encode them into numerical values using techniques such as one-hot encoding or label encoding.
6. **Data Balancing (if applicable):**
   - If the dataset is imbalanced, where certain classes of structural damage are underrepresented, apply techniques such as oversampling, undersampling, or synthetic data generation to balance the dataset.
7. **Data Partitioning:**

- Split the preprocessed dataset into training, validation, and test sets for model development, evaluation, and testing purposes, respectively.

8. **Data Augmentation (optional):**
   - Augment the dataset by generating additional samples through transformations such as rotation, translation, or scaling.
   - Data augmentation can help improve model generalization and robustness, especially when dealing with limited data.
9. **Data Representation:**
   - Represent the preprocessed data in a suitable format for AIPR algorithms to process.
   - Data representation may involve converting the data into antigenic form, where each instance is represented as an antigen with specific features.

Data Representaion: Antigen Representation:

- In AIPR, antigens represent patterns or instances from the dataset.
- Each antigen encapsulates the features or attributes of a data point.
- Antigens can be represented as vectors, matrices, or any other suitable data structure depending on the nature of the data.
- For structure damage classification, antigens may include various features such as material properties, geometric characteristics, environmental conditions, etc.

Classification:

- Given a new antigen (representing a new instance of structural damage), classify it by evaluating its similarity to the antibodies.
- Measure the similarity between the antigen and each antibody using a predefined similarity metric (e.g., Euclidean distance or cosine similarity).
- Classify the antigen based on the classification of the antibody with the highest affinity.
- The classification result may include the predicted class label and the confidence level (affinity) associated with the classification.

Conclusion: -

_____

_____

Questions:

1. Describe AIS?
2. Explain Data Representation in AIS ?
3. Define Data Cleaning & Feature Extraction?
4. Why their is need of AIS?

Assignment No.2 (Part B)

Title: Implement DEAP (Distributed Evolutionary Algorithms) using Python

Aim: Implement DEAP (Distributed Evolutionary Algorithms) using Python

- **Outcome:** At end of this experiment, student will be able understand DEAP
- **Hardware Requirement:** Computer System, Linux(Ubuntu)
- **Software Requirement:** PyCharm IDE

Theory:

- DEAP is a novel evolutionary computation framework for rapid prototyping and testing of ideas. It seeks to make algorithms explicit and data structures transparent. It works in perfect harmony with parallelization mechanisms such as multiprocessing and SCOOP.

DEAP includes the following features:

- Genetic algorithm using any imaginable representation
  - List, Array, Set, Dictionary, Tree, Numpy Array, etc.
- Genetic programming using prefix trees
  - Loosely typed, strongly typed
  - Automatically defined functions
- Evolution strategies (including CMA-ES)
- Multi-objective optimisation (NSGA-II, NSGA-III, SPEA2, MO-CMA-ES)
- Co-evolution (cooperative and competitive) of multiple populations
- Parallelization of the evaluations (and more)
- Hall of Fame of the best individuals that lived in the population
- Checkpoints that take snapshots of a system regularly
- Benchmarks module containing most common test functions
- Genealogy of an evolution (that is compatible with NetworkX)
- Examples of alternative algorithms: Particle Swarm Optimization, Differential Evolution, Estimation of **Distribution Algorithm**

Overview

If you are used to any other evolutionary algorithm framework, you'll notice we do things differently with DEAP. Instead of limiting you with predefined types, we provide ways of creating the appropriate ones. Instead of providing closed initializers, we enable you to customize them as you wish. Instead of suggesting unfit operators, we explicitly ask you to choose them wisely. Instead of implementing many sealed algorithms, we allow you to write the ones that fit all your needs. This tutorial will present a quick overview of what DEAP is all about along with what every DEAP program is made of.

Types

The first thing to do is to think of the appropriate type for your problem. Then, instead of looking in the list of available types, DEAP enables you to build your own. This is done with the creator module. Creating an appropriate type might seem overwhelming but the creator makes it very easy. In fact, this is usually done in a single line. For example, the following creates a FitnessMin class for a minimization problem and an Individual class that is derived from a list with a fitness attribute set to the just created fitness.

```
from deap import base, creator
creator.create("FitnessMin", base.Fitness, weights=(-1.0,))
creator.create("Individual", list, fitness=creator.FitnessMin)
```

That's it. More on creating types can be found in the Creating Types tutorial.

Initialization

Once the types are created you need to fill them with sometimes random values or sometime guessed ones. Again, DEAP provides an easy mechanism to do just that. The Toolbox is a container for tools of all sorts including initializers that can do what is needed of them. The following takes on the last lines of code to create the initializers for individuals containing random floating point numbers and for a population that contains them.

```
import random
from deap import tools

IND_SIZE = 10

toolbox = base.Toolbox()
toolbox.register("attribute", random.random)
toolbox.register("individual", tools.initRepeat, creator.Individual,
                 toolbox.attribute, n=IND_SIZE)
toolbox.register("population", tools.initRepeat, list, toolbox.individual)
```

- This creates functions to initialize populations from individuals that are themselves initialized with random float numbers. The functions are registered in the toolbox with their default arguments under the given name. For example, it will be possible to call the function toolbox.population() to instantly create a population. More initialization methods are found in the Creating Types tutorial and the various Examples.

Operators

- Operators are just like initializers, except that some are already implemented in the tools module. Once you've chosen the perfect ones, simply register them in the toolbox. In addition, you must create your evaluation function. This is how it is done in DEAP.

```
def evaluate(individual):
    return sum(individual),
```

```
toolbox.register("mate", tools.cxTwoPoint)
toolbox.register("mutate", tools.mutGaussian, mu=0, sigma=1, indpb=0.1)
toolbox.register("select", tools.selTournament, tournsize=3)
toolbox.register("evaluate", evaluate)
```

- The registered functions are renamed by the toolbox, allowing generic algorithms that do not depend on operator names. Note also that fitness values must be iterable, that is why we return a tuple in the evaluate function. More on this in the Operators and Algorithms tutorial and Examples. Algorithms Now that everything is ready, we can start to write our own algorithm. It is usually done in a main function. For the purpose of completeness, we will develop the complete generational algorithm.

```
def main():
    pop = toolbox.population(n=50)
    CXPB, MUTPB, NGEN = 0.5, 0.2, 40

    # Evaluate the entire population
    fitnesses = map(toolbox.evaluate, pop)
    for ind, fit in zip(pop, fitnesses):
        ind.fitness.values = fit

    for g in range(NGEN):
        # Select the next generation individuals
        offspring = toolbox.select(pop, len(pop))
        # Clone the selected individuals
        offspring = map(toolbox.clone, offspring)

        # Apply crossover and mutation on the offspring
        for child1, child2 in zip(offspring[::2], offspring[1::2]):
            if random.random() < CXPB:
                toolbox.mate(child1, child2)
                del child1.fitness.values
                del child2.fitness.values

        for mutant in offspring:
            if random.random() < MUTPB:
                toolbox.mutate(mutant)
                del mutant.fitness.values

        # Evaluate the individuals with an invalid fitness
        invalid_ind = [ind for ind in offspring if not ind.fitness.valid]
        fitnesses = map(toolbox.evaluate, invalid_ind)
        for ind, fit in zip(invalid_ind, fitnesses):
            ind.fitness.values = fit
```

```
    # The population is entirely replaced by the offspring
    pop[:] = offspring

return pop
```

It is also possible to use one of the four algorithms readily available in the algorithms module, or build from some building blocks called variations also available in this module.

Installation Requirements

DEAP is compatible with Python 2.7 and 3.4 or higher. The computation distribution requires SCOOP. CMA-ES requires Numpy, and we recommend matplotlib for visualization of results as it is fully compatible with DEAP's API.

Install DEAP

We encourage you to use easy_install or pip to install DEAP on your system. Linux package managers like apt-get, yum, etc. usually provide an outdated version.

```
easy_install deap
```

or

```
pip install deap
```

If you wish to build from sources, download or clone the repository and type:

```
python setup.py install
```

Conclusion: -

_____

_____

Questions:

1. What is **DEAP** and what are its key features in evolutionary computation?
2. Explain the role of the **creator module** in DEAP. How are **Fitness** and **Individual** classes created?
3. What is a **Toolbox** in DEAP? How is it used for **initialization, operators (crossover, mutation, selection), and evaluation functions**?
4. Describe the working of a **genetic algorithm in DEAP**, including steps like initialization, evaluation, selection, crossover, mutation, and replacement.

Assignment No. 3 (Part B)

Title: Design and Develop a Distributed Application to Find the Coolest/Hottest Year Using Map Reduce.

Aim: To design and develop a distributed application using **MapReduce** to analyze large-scale weather data and determine the **coolest and hottest year** from the available dataset.

2. Objective

- To understand distributed data processing using **Hadoop MapReduce**
- To process large weather datasets efficiently
- To compute yearly average temperatures
- To identify the year with:
    - Highest average temperature (Hottest Year)
    - Lowest average temperature (Coolest Year)

3. Problem Statement

Weather datasets collected over several years contain temperature readings. Since the data size is very large, processing it on a single machine is inefficient.

Using **MapReduce**, develop a distributed application that:

1. Extracts temperature data year-wise
2. Computes average temperature for each year
3. Identifies the coolest and hottest year

4. Theory

4.1 Distributed Computing

Distributed computing divides computation across multiple machines to process large datasets efficiently.

4.2 Hadoop Ecosystem

- **HDFS (Hadoop Distributed File System)** – Stores large datasets
- **MapReduce** – Programming model for distributed data processing

4.3 MapReduce Model

MapReduce consists of two main phases:

1. Map Phase

- Input: Raw weather records

- Output: Key-value pairs (Year, Temperature)

2. Reduce Phase

- Input: (Year, List of Temperatures)
- Output: (Year, Average Temperature)

Working Flow

Weather Dataset → HDFS → Mapper → Shuffle & Sort → Reducer → Output (Year-wise Avg Temp) → Find Coolest & Hottest Year

Software Requirements

- Java JDK 8+
- Hadoop (2.x or 3.x)
- Eclipse / IntelliJ IDEA
- Linux / Ubuntu (preferred)
- Internet connection (to download weather dataset)

Hardware Requirements

- Minimum 4GB RAM
- 50GB Storage
- Multi-core Processor

Dataset

Weather data can be downloaded from:

- NOAA (National Oceanic and Atmospheric Administration)
- Kaggle weather datasets
- Government meteorological websites

Sample Data Format

2001,01,15,32
2001,02,20,35
2002,03,10,28
2002,04,12,40

Format:

Year, Month, Day, Temperature

Algorithm

Step 1: Mapper

1. Read input line
2. Extract Year and Temperature
3. Emit (Year, Temperature)

Step 2: Reducer

1. Receive (Year, List of Temperatures)
2. Compute average temperature
3. Emit (Year, AvgTemperature)

Step 3: Post-processing

1. Compare yearly averages
2. Find:
    - Maximum average → Hottest Year
    - Minimum average → Coolest Year

Program Code (Java MapReduce)

9.1 Mapper Class

```
import java.io.IOException;
import org.apache.hadoop.io.*;
import org.apache.hadoop.mapreduce.Mapper;

public class WeatherMapper extends Mapper<Object, Text, Text, FloatWritable> {

   private Text year = new Text();
   private FloatWritable temperature = new FloatWritable();

   public void map(Object key, Text value, Context context)
         throws IOException, InterruptedException {

      String[] fields = value.toString().split(",");

      if (fields.length == 4) {
         year.set(fields[0]);
         temperature.set(Float.parseFloat(fields[3]));
         context.write(year, temperature);
      }
   }
}
```

9.2 Reducer Class

```
import java.io.IOException;
```

```
import org.apache.hadoop.io.*;
import org.apache.hadoop.mapreduce.Reducer;

public class WeatherReducer extends Reducer<Text, FloatWritable, Text, FloatWritable> {

   public void reduce(Text key, Iterable<FloatWritable> values, Context context)
         throws IOException, InterruptedException {

      float sum = 0;
      int count = 0;

      for (FloatWritable val : values) {
         sum += val.get();
         count++;
      }

      float average = sum / count;
      context.write(key, new FloatWritable(average));
   }
}
```

9.3 Driver Class

```
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.*;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class WeatherDriver {

   public static void main(String[] args) throws Exception {

      Configuration conf = new Configuration();
      Job job = Job.getInstance(conf, "Weather Analysis");

      job.setJarByClass(WeatherDriver.class);
      job.setMapperClass(WeatherMapper.class);
      job.setReducerClass(WeatherReducer.class);

      job.setOutputKeyClass(Text.class);
      job.setOutputValueClass(FloatWritable.class);

      FileInputFormat.addInputPath(job, new Path(args[0]));
      FileOutputFormat.setOutputPath(job, new Path(args[1]));

      System.exit(job.waitForCompletion(true) ? 0 : 1);
   }
```

}

Execution Steps

Step 1: Compile

javac -classpath `hadoop classpath` -d weather_classes *.java
jar -cvf weather.jar -C weather_classes/ .

Step 2: Upload Data to HDFS

hdfs dfs -mkdir /weather
hdfs dfs -put weather.csv /weather

Step 3: Run MapReduce Job

hadoop jar weather.jar WeatherDriver /weather /weather_output

Step 4: View Output

hdfs dfs -cat /weather_output/part-r-00000

Sample Output

2001 33.5
2002 34.2
2003 31.8

From this:

- Hottest Year → 2002
- Coolest Year → 2003

Conclusion:

Questions:

1. What is MapReduce?
2. What is the difference between HDFS and MapReduce?
3. What happens during shuffle and sort phase?
4. Why is distributed processing needed for weather data?

Assignment No. 4 (Part B)

Title: Implement Ant colony optimization by solving the Traveling salesman problem using python Problem statement- A salesman needs to visit a set of cities exactly once and return to the original city. The task is to find the shortest possible route that the salesman can take to visit all the cities and return to the starting city.

Aim: Ant colony optimization using Traveling salesman problem

Problem Statement: Implement Ant colony optimization by solving the Traveling salesman problem using python

Problem statement- A salesman needs to visit a set of cities exactly once and return to the original city. The task is to find the shortest possible route that the salesman can take to visit all the cities and return to the starting city.

Theory: Ant Colony Optimization:

• The algorithmic world is beautiful with multifarious strategies and tools being developed round the clock to render to the need for high-performance computing. In fact, when algorithms are inspired by natural laws, interesting results are observed.

• Evolutionary algorithms belong to such a class of algorithms. These algorithms are designed so as to mimic certain behaviours as well as evolutionary traits of the human genome.

• Moreover, such algorithmic design is not only constrained to humans but can be inspired by the natural behaviour of certain animals as well. The basic aim of fabricating such methodologies is to provide realistic, relevant and yet some low cost solutions to problems that are hitherto unsolvable by conventional means.

• Different optimization techniques have thus evolved based on such evolutionary algorithms and thereby opened up the domain of metaheuristics. Metaheuristic has been derived from two Greek words, namely, Meta meaning one level above and heuriskein meaning to find.

• Algorithms such as the Particle Swarm Optimization (PSO) and Ant Colony Optimization (ACO) are examples of swarm intelligence and metaheuristics. The goal of swarm intelligence is to design intelligent multi-agent systems by taking inspiration from the collective behaviour of social insects such as ants, termites, bees, wasps, and other animal societies such as flocks of birds or schools of fish. Background:

• Ant Colony Optimization technique is purely inspired from the foraging behaviour of ant colonies, first introduced by Marco Dorigo in the 1990s.

• Ants are eusocial insects that prefer community survival and sustaining rather than as individual species.

• They communicate with each other using sound, touch and pheromone. Pheromones are organic chemical compounds secreted by the ants that trigger a social response in members of same species.

• These are chemicals capable of acting like hormones outside the body of the secreting individual, to impact the behaviour of the receiving individuals.

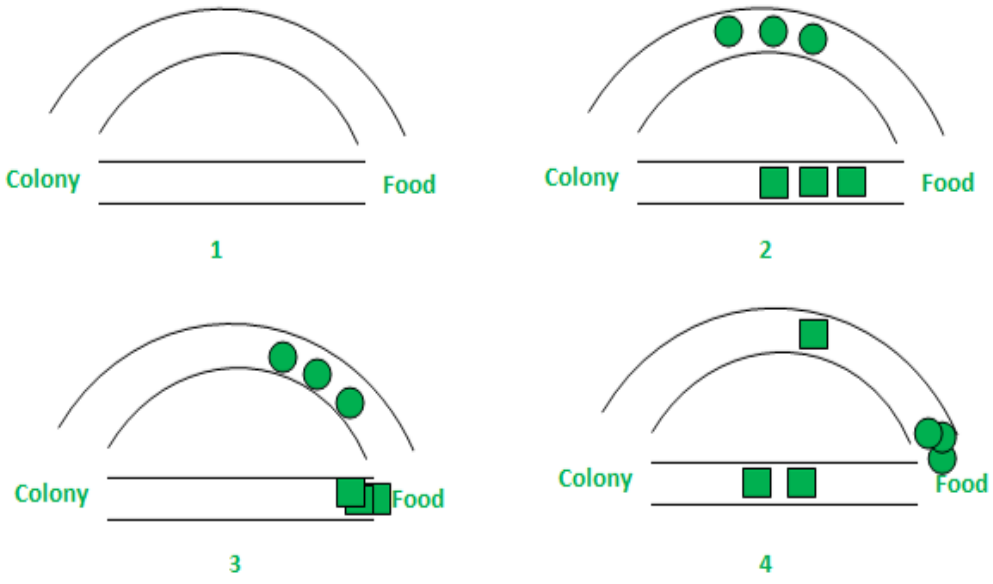
• Since most ants live on the ground, they use the soil surface to leave pheromone trails that may be followed (smelled) by other ants.

• Ants live in community nests and the underlying principle of ACO is to observe the movement of the ants from their nests in order to search for food in the shortest possible path. Initially, ants start to move randomly in search of food around their nests.

• This randomized search opens up multiple routes from the nest to the food source. Now, based on the quality and quantity of the food, ants carry a portion of the food back with necessary pheromone concentration on its return path.

• Depending on these pheromone trials, the probability of selection of a specific path by the following ants would be a guiding factor to the food source. Evidently, this probability is based on the concentration as well as the rate of evaporation of pheromone.

• It can also be observed that since the evaporation rate of pheromone is also a deciding factor, the length of each path can easily be accounted for.In the above figure, for simplicity, only two possible paths have been considered between the food source and the ant nest. • The stages can be analyzed as follows:



• Stage 1: All ants are in their nest. There is no pheromone content in the environment. (For algorithmic design, residual pheromone amount can be considered without interfering with the probability)

• Stage 2: Ants begin their search with equal (0.5 each) probability along each path. Clearly, the curved path is the longer and hence the time taken by ants to reach food source is greater than the other.

• Stage 3: The ants through the shorter path reaches food source earlier. Now, evidently they face with a similar selection dilemma, but this time due to pheromone trail along the shorter path already available, probability of selection is higher.

• Stage 4: More ants return via the shorter path and subsequently the pheromone concentrations also increase. Moreover, due to evaporation, the pheromone concentration in the longer path reduces, decreasing the probability of selection of this path in further stages. Therefore, the whole colony gradually uses the shorter path in higher probabilities. So, path optimization is attained.

Conclusion :----------

Questions:

1. What is TSP?
2. Why is TSP NP-Hard?
3. What is Ant Colony Optimization?
4. What is pheromone evaporation?
5. What is the role of alpha and beta?
6. Difference between Genetic Algorithm and ACO?

Assignment No. 5 (Part B)

Title: Create and Art with Neural style transfer on given image using deep learning.

Aim: To implement **Neural Style Transfer (NST)** using Deep Learning techniques to generate artistic images by combining the **content of one image** with the **style of another image**.

Problem Statement

Create artistic images using Neural Style Transfer by:

- Taking a **Content Image**
- Taking a **Style Image**
- Generating a new image that preserves:
    - Content structure from the content image
    - Artistic style (colors, textures, brush strokes) from the style image

Theory

5.1 What is Neural Style Transfer?

Neural Style Transfer (NST) is a deep learning technique that:

- Extracts **content features** from one image
- Extracts **style features** from another image
- Combines them to generate a new artistic image

5.2 Convolutional Neural Networks (CNN)

NST uses a **pre-trained CNN (VGG-19)** to extract features.

CNN Layers:

- Lower layers → Detect edges and textures
- Middle layers → Detect shapes
- Higher layers → Detect complex objects

5.3 Content Representation

Content loss measures difference between:

- Feature maps of content image
- Feature maps of generated image

Formula:

$$L_{content} = \frac{1}{2}\sum(F_{ij} - P_{ij})^2$$

5.4 Style Representation

Style is represented using **Gram Matrix**:

$$G = FF^T$$

Style loss measures difference between:

- Gram matrix of style image
- Gram matrix of generated image

5.5 Total Loss

$$L_{total} = \alpha L_{content} + \beta L_{style}$$

Where:

- α → Content weight
- β → Style weight

• Neural Style Transfer is a deep learning technique that combines the content of one image (content image) with the style of another image (style image) to create a new image (generated image). • It leverages convolutional neural networks (CNNs) to extract content and style features from the input images and optimize a generated image to minimize the content distance from the content image and the style distance from the style image simultaneously.

Key Steps of NST: • Choose Content and Style Images: Select a content image (the image whose content you want to retain) and a style image (the image whose artistic style you want to apply). • Preprocess Images: Preprocess the content and style images to prepare them for feeding into a CNN. • Define Loss Functions: Define content loss and style loss functions to measure the difference between the features of the generated image, content image, and style image. • Optimization: Use gradient descent or other optimization techniques to update the generated image to minimize the combined content and style loss. • Generate Artistic Image: Generate the final artistic image by optimizing the generated image. Conclusion: Thus, we have successfully implemented a program for Neural style transfer.

Distributed Computing Perspective

Neural Style Transfer requires:

- Heavy computation
- Large matrix operations
- GPU acceleration

NST can be implemented in distributed environments using:

- Google Colab (GPU)
- AWS EC2
- Azure ML
- TensorFlow Distributed Strategy
- PyTorch Distributed Data Parallel

This experiment demonstrates AI workload processing in distributed computing systems.

Conclusion :----------

Questions:

1. What is Neural Style Transfer?
2. Why is VGG19 used?
3. What is Gram Matrix?
4. Difference between content loss and style loss?